BsC Devops

Group D

Oriol Grau Moragues - s25137@itu.dk

Mohamed Karam Haybout - mhay@itu.dk

Anton Thejsen - antt@itu.dk

Jordan Cherry - s25121@itu.dk

Madeleine Jakobsen - majak@itu.dk

Tim Vogensen Hounsgaard - thou@itu.dk

System's perspective

Design and Architecture

Our system consists of the following.

Component	Count	Technologies
Reserved IP	1	DigitalOcean Reserved IP
Load Balancers	2	Droplet & nginx
Web + API	2	Droplet, Docker, Grafana Alloy
Monitoring	1	Droplet, Loki, Promethus & Grafana
Managed Database	1	Digitalocean Managed Database (Postgres)

Reserved IP Address

A single reserved IP Address to serve as a stable public entrypoint for all inbound traffic. It is satically assigned and does not change regardless of infrastructure state. At any given time, the reserved IP is mapped to one of the two load balancer nodes. In the event a load balancer goes offline the IP can be remapped to the second load balancer, ensuring continuing service.

Load Balancer

Two load balancer nodes running on DigitalOcean Droplets running nginx. Both nodes are keepalives, which continouously monitors the health of the active node and orchestrates the reassingment of the reserved IP to the secondary node if failure is detected. Each load balancer functions as a reverse proxy. Incoming requests are forwarded to any of the Web/Api Servers depending on the IP-Hashing, ensuring deterministic behaviour. (Is this correct? - Anton)

Web and API Servers

Two droplets serve as the application layer, each running both the Web application image and the API Image, meaning our system maintains two concurrent instances of each service, providing redundancy and enabling load distribution across both nodes. Additionally each Web & API server runs Grafana Alloy, which is responsible for collecting and forwarding logs and metrics to our centralised monitoring server.

Monitoring Server

A dedicated droplet hosts the monitoring stack, which centralises the collection and storage of logs and data from the applications.

Managed Database

Application data is persisted in a DigitalOcean Managed Database running PostreSQL. Both Web and API server instances are connected to the shared database.

Dependencies

State of the System

Our systems remain highly operational and secure and has been guarded against the introduction of weak code and dangerous vulnerabilities via the enforcement of static analysis and quality assessments in our CICD pipeline. We make use of CodeQL, SonarQube and Megalinters.

Currently we stand at 94 reported issues by CodeQL, all of them reporting poor coding conventions. We had closed a total of 42 other issues that were a mix of critical and mediocre coding issues.

SonarQube reports an A grade over Security, Reliability and Maintainability. The main reported issues within these are regarding a 3% technical debt ratio within Maintainability. While it does not impact the grade, it should not be ignored as technical debt can easily grow out of control.

There are also a reported 20 reliability issues and 797 maintainability issues, mostly flagged as “Medium” severity involving the usage of proper coding conventions and type safety.

Overall the system is safe and could use more improvements but nothing critical can cause it to shut down. If any new code were introduced then our tools are set up to block any PRs when highly dangerous issues are detected.

Process' perspective

CI/CD Pipeline, Stages and Tools

Our CI/CD pipeline has grown to become very comprehensive and aims to perform its tasks as quick as possible in order to minimize developer wait time and ensure code can be delivered properly towards production.

The system comprises of the following major stages:

Build & Test

1. Build and upload build artifacts
2. Parallelize the following jobs using build artifacts:
    - CodeQL Analysis
        - In hindsight, it is possible to move this job into a “Code Quality” workflow alongside the MegaLinter stage and make it dependent on the uploaded build artifacts for better architectural readability
    - Check Migrations
        - Ensures the developer doesn't forget to create a new migration if they have made changes to the entity model
    - Test Suite
        - Unit, Integration and End2End are all run in parallel for quickest job time

MegaLinter

1. Download Megalinters Docker image
    - The biggest bottleneck remains here due to GitHub runners needing to redownload the image on every job run. Ideally we would use our own selfhosted runner to minimize download time
2. Run static analysers
3. Upload result artifacts to GH PR
4. Commit and Push auto-fix to GH PR
    - While useful, this will cause both the BuildTest and MegaLinter stages to restart, thus wasting time. We did not get around to figuring out a way to optimize this issue while retaining the benefits of auto-fixes.

Docker Build & Publish

1. Build and Publish web and api images in parallel:
    - Build Image
    - Run Docker Scout vulnerability scan
    - Push image to Docker Hub

Smoke Deploy & Deploy

On either a PR to Main, or push to main, a deploy will happen to either the staging server or production server

1. Build migration bundle
2. Install Ansible & Tofu
3. Run Deploy IaC

How do you monitor your systems and what precisely do you monitor? - Madeleine

How do you monitor your systems and what precisely do you monitor? -Madeleine

The team has used a combination of Prometheus and Grafana to monitor the project. Prometheus is used to collect metrics, while Grafana is used to visualize the data into something that can easily be understood. We currently monitor the number of requests that certain API endpoints get, such as : the 'post' endpoint for messages. The Prometheus and Grafana systems were added with PR#26. In addition to this, we monitor the amount of time each API request takes using histograms, PR#38.

What do you log in your systems and how do you aggregate logs? -Madeleine

We logs any errors that occur when processing API requests, as well as when an API request is successful. Logs often contain variables such as status codes, usernames, the type of API request, and more... We also log all requests for the project's frontend web application. All logs can be found on the projects Grafana log page, with the logs being collected with Prometheus.

Brief description of how your security hardened your systems -Tim and Oriol

The System has been hardened with a fire wall and a proxy server so all traffic coming to the web/api app goes through a proxy server. all communication between user and proxy, and proxy and apps are delivered through HTTPS using TLS encryption. We updated the docker images to use a hardened image for security, and to secure we not introducing new security vulnerabilities CodeQL and Docker Scout was put in place in the CI pipeline to sniff out security vulnerabilities. For local development to store secrets locally we used .env files so our secrets weren't shared online.

How do you handle availability and scaling in your systems? -Jordan

Docker Swarm and rolling upgrades
Docker Swarm was introduced to manage updates without taking the system offline by stopping and restarting the containers each time through docker-compose up. Swarm allowed for the updates to be applied incrementally, bringing up new containers with the updated images before stopping the old ones. This allowed for users and the simulator to have no downtime as it was deploying, which was particularly important to prevent simulator issues.

Staging
A staging branch was created to validate new changes before deployment, where there is no risk of affecting the production environment. The testing of these changes in a separate environment reduces the chance of needing to rollback on the main branch regarding integration issues.

Health check and rollback
After each deployment, the pipeline curls the web server image several times. If the service fails to respond, the pipeline automatically redeploys the previous commit’s image, restoring the previous state automatically.

Database migration
The deploy workflow automatically detects if any EF Core migration files changed between the commits and applies them before bringing up new containers. This keeps the database schema in sync with the application.

Variable number of instances
By using infrastructure as code, we can provision new instances and add them to the system easily. As the user base grows, new servers can be spin up without additional work other than specifying the number of instances and re-

running the IaC tools. Furthermore, our load balancing setup reduces the chances of overloading specific web servers and the load balancers are monitored with keepalived, which switches the active load balancer in case the current one fails.

Reflection' perspective

Creating staging - Madeleine

We wanted to prevent downtime in our application, that could occur if a push broke something in the production. We created a staging branch that mirrors our Main branch closely in order to avoid those downtimes. The staging branch is connected to a staging droplet inside DigitalOcean. This branch was created for us to push new changes to, without impacting Main. Allowing us to see how Main would be impacted by the changes in a safe environment, and catch any major errors before they are deployed. There are multiple workflows that run on the staging branch, these include linting and testing steps, this can be found in the PR#09. This is relevant in regards to Operation and Maintenance, as the staging branch helps keep the application healthy and running.

Refactoring user indexing - Madeleine

We decided to use the project files from the course BDSA as the starting point for this project, this meant that any existing errors in the previous project would be also present in this project. This lead to the team getting many errors once the simulator began to run, specifically with the creation of users. The project we used indexed users in a very inefficient way, and meant that if two users were created at the same time, they could both share the same index, leading to users being overwritten in the database. The system could not handle asynchronous tasks.
The team fixed the issue by refactoring the way the user indexing worked, letting the database automatically assign indexes instead of doing it manually. We learned that it is important to have very in-depth testing, to see how well an application handles multiple requests and tasks at once. Stress testing can also be a good way to find failures in the system.

Reflect and describe what was the "DevOps" style of your work. For example, what did you do differently to previous development projects and how did it work?

Overall the project introduced some unique challenges and new ways of working we have not been to used too. instead of having a system that just needed to work completely when handing it in. Now we had a system that needed to be live and healthy all the time.
We introduced a lot of new ways of working to accommodate this criteria. We created a staging environment that worked like production where we could test our code live before it hit production. We created more in depth CI pipeline to validate the quality of our code together with verifying we aren't introducing vulnerabilities when adding or updating packages.

Smoke Deploy issues

After we had implemented docker swarm and its rolling upgrade functionality in commit #f15cc59, we encountered inconsistent issues regarding the smoke deploy step in our CICD. Occasionally a smoke deploy would hang on either docker swarm activity check, applying migrations, or rolling upgrade step.

We had made multiple fixes but the issue would keep resurfacing later. It was only after inspecting the staging server that we discovered the issue lied in the limited resources the server provides. Maxed out CPU and RAM made any other operation besides the already running docker containers impossible.

We managed to fix this by implementing a resource guard in commit 0620a6a to free up the servers resources during a deployment but this solution unfortunately sacrifices the full uptime that we aimed to get via rolling upgrades.

Overall this entire endeavor showed us the importance of running workflows locally to validate their results instead of constantly committing code to see if it solves the problem. This approach should be aimed towards for the sake of maintainability and operation.

Workflow Concurrency issues - Jordan

A concurrency guard was implemented to solve issues regarding deploys running simultaneously, as this would cause server overload and conflicting migrations. This concurrency guard was implemented differently in production and

staging, with the production version using cancel-in-progress = false to avoid silently skipping deployments, and cancel-in-progress = true for staging, as only the most up-to-date version mattered. This is evident in the commit “ab30aa6”.

The key lesson learnt was that even with a ci/cd pipeline that can work in processing a deployment, additional guardrails are still necessary for problems with multiple users or commits.

Use of Generative AI -Tim

Generative AI has been used throughout the project to help us, complete the weekly tasks. AI has been used as a debug tool / sparring partner, when a person was stuck. AI has also been used to generate a starting point for task with new technologies. The group has their own preferences in AIs so Gemini, Chatgpt, Claude and Copilot has been used during the project All have been marked as co-author when used. Pro and cons follows in the use of AI. For our purpose it has been a great tool that has saved us countless hours searching on the web, however this "easy" way could also be a hinderance in the sense it might halluciate a solution to our problem that does not work, and us taking the "easy" made harder to catch that. All in all it can be good, but we have found limiting the amount of usage is benefitial.